Open in app ↗ Sign up Sign in

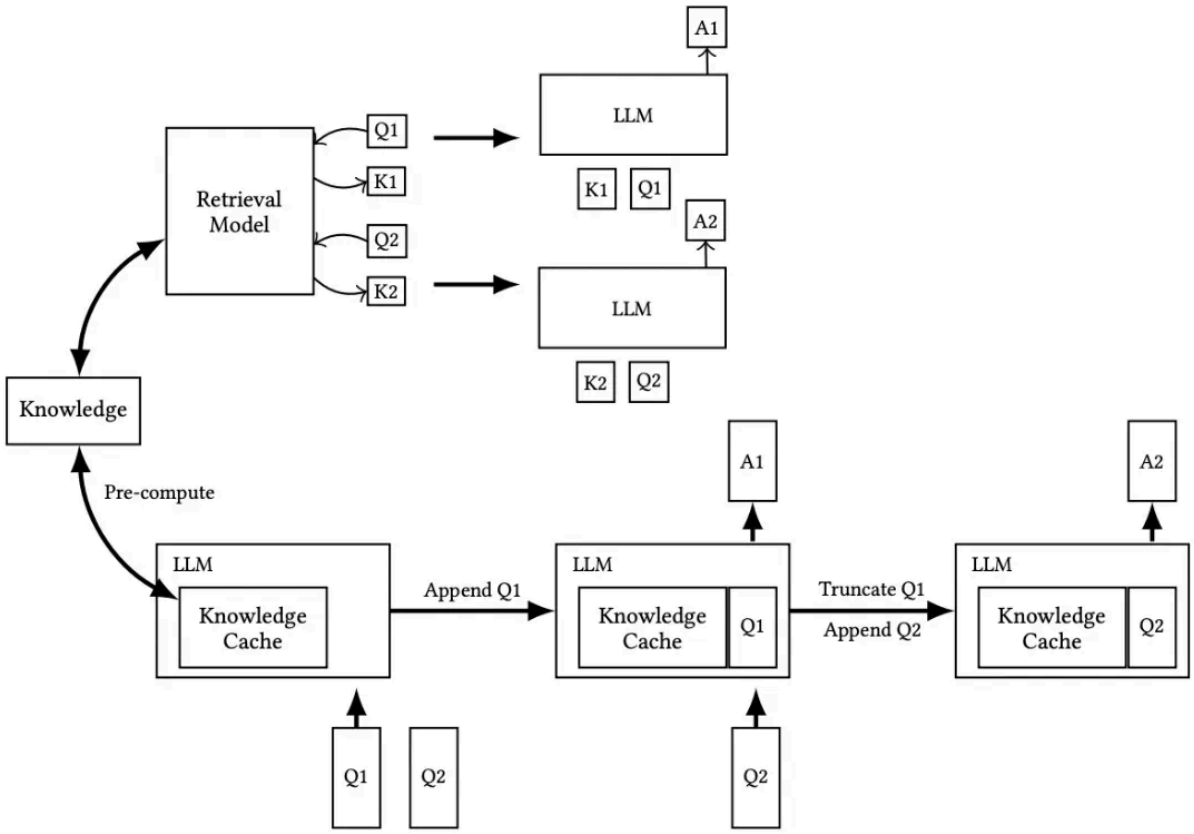
Medium Search Write

GoPenAI

# Cache-Augmented Generation (CAG) in LLMs: A Step-by-Step Tutorial

Ronan Takizawa Follow 5 min read · Jan 2, 2025

299 4




Full Code (Make sure to leave the Original Repo a Star!) ⭐

**Retrieval-augmented generation (RAG)** is a powerful method to connect external knowledge bases to an LLM and fetch context each time a user asks a question, but it can slow down the LLM's performance due to its retrieval latency.

**Cache-augmented generation (CAG)** offers a faster alternative; instead of performing real-time retrieval, it *preloads* your relevant documents into the model's context and stores that inference state — also known as a Key-Value (KV) cache. This approach eliminates retrieval latencies, allowing the model to access preloaded information instantly for faster and more efficient responses.

For a more technical explanation of CAG, check out this article.

In this tutorial, we will show how to build a simple **CAG** setup to embed all your knowledge upfront, quickly answer multiple user queries, and reset the cache without reloading the entire context each time.

### Prerequisites

1. A HuggingFace account and a HuggingFace access token

2. A document.txt file with sentences about yourself.

### Project Setup

We import the essential libraries:

- `torch` for PyTorch.
- `transformers` for Hugging Face.
- `DynamicCache` for storing the model's key-value states.

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers.cache_utils import DynamicCache
import os
```

### Generate Function

We'll next define the `generate` function.

The `generate` function handles token-by-token generation with the cached knowledge using greedy decoding.

Greedy decoding is a simple text generation method where, at each step, the token with the highest probability (maximum value in the logits) is selected as the next token.

We pass in these inputs:

- `model` : The LLM, which with me Mistral-7B for this tutorial.
- `input_ids` : A tensor containing the tokenized input sequence.
- `past_key_values` : The core component of the CAG. A cache of previously computed attention values is used to speed up inference by avoiding recomputation.
- `max_new_tokens` : The maximum number of new tokens to generate. The default is 50.

The function operates in a loop that iterates up to `max_new_tokens` times or terminates early if an end-of-sequence token (if configured) is generated.

At each iteration:

- The model processes the current input tokens along with the cached `past_key_values` , producing logits for the next token.
- The logits are analyzed to identify the token with the highest probability using greedy decoding.
- This new token is appended to the output sequence, and the cache ( `past_key_values` ) is updated to include the current context.
- The newly generated token becomes the input for the next iteration.

```
def generate(model, input_ids: torch.Tensor, past_key_values, max_new_tokens: in
    device = model.model.embed_tokens.weight.device
    origin_len = input_ids.shape[-1]
    input_ids = input_ids.to(device)
    output_ids = input_ids.clone()
    next_token = input_ids

    with torch.no_grad():
        for _ in range(max_new_tokens):
            out = model(
                input_ids=next_token,
                past_key_values=past_key_values,
                use_cache=True
            )
            logits = out.logits[:, -1, :]
            token = torch.argmax(logits, dim=-1, keepdim=True)
            output_ids = torch.cat([output_ids, token], dim=-1)
            past_key_values = out.past_key_values
            next_token = token.to(device)

            if model.config.eos_token_id is not None and token.item() == model.c
                break
    return output_ids[:, origin_len:]
```

## DynamicCache Setup

Next, we'll define the `get_kv_cache` function that prepares a reusable key-value cache for a transformer model's attention mechanism and the `clean_up` function that cleans the key-value cache by removing unnecessary entries to ensure that you can answer multiple independent questions without "polluting" the cache.

Get Ronan Takizawa's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

`get_kv_cache` passes a prompt (in our case, the knowledge from `document.txt`) through the model once, creating a KV cache that records all the hidden states from each layer.

`get_kv_cache` passes in these inputs:

- `model` : The transformer model used for encoding the prompt.
- `tokenizer` : Tokenizer to convert the prompt into token IDs.
- `prompt` : A string input is used as the prompt.

and returns an object of the type `DynamicCache`.

The `get_kv_cache` function first tokenizes the provided prompt using the tokenizer, converts it into input IDs, and then initializes an `DynamicCache` object to store key-value pairs, and then performs a forward pass through the model with caching enabled ( `use_cache=True` ). This populates the cache with the key-value pairs resulting from the model's computation.

The `clean_up` trims a `DynamicCache` object to match the original sequence length by removing any additional tokens added during processing. For each layer of the cache, it slices both the key and value tensors to retain only the first `origin_len` tokens along the sequence dimension.

```
def get_kv_cache(model, tokenizer, prompt: str) -> DynamicCache:
    device = model.model.embed_tokens.weight.device
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)
    cache = DynamicCache()

    with torch.no_grad():
        _ = model(
            input_ids=input_ids,
```

```
                past_key_values=cache,
                use_cache=True
            )
        return cache

    def clean_up(cache: DynamicCache, origin_len: int):
        for i in range(len(cache.key_cache)):
            cache.key_cache[i] = cache.key_cache[i][:, :, :origin_len, :]
            cache.value_cache[i] = cache.value_cache[i][:, :, :origin_len, :]
```

### Load LLM (Mistral)

Now we'll load the Mistral-7B model, and load the tokenizer and model in full precision or half precision (FP16) on GPU if available.

Remember to input `YOUR_HF_TOKEN` with your unique HuggingFace Token.

```
model_name = "mistralai/Mistral-7B-Instruct-v0.1"
tokenizer = AutoTokenizer.from_pretrained(model_name, token="YOUR_HF_TOKEN", tru
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto",
    trust_remote_code=True,
    token="YOUR_HF_TOKEN"
)
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
print(f"Loaded {model_name}.")
```

### Create a Knowledge Prompt from document.txt

Next, we'll read `document.txt` , which you can fill with information about yourself. For this tutorial, `document.txt` contains information about me (Ronan Takizawa).

Here we construct a simple system prompt embedding with the doc information and pass it to `get_kv_cache` to generate the KV cache.

```
with open("document.txt", "r", encoding="utf-8") as f:
    doc_text = f.read()

system_prompt = f"""
<|system|>
You are an assistant who provides concise factual answers.
<|user|>
Context:
{doc_text}
Question:
""".strip()

ronan_cache = get_kv_cache(model, tokenizer, system_prompt)
origin_len = ronan_cache.key_cache[0].shape[-2]
print("KV cache built.")
```

### Ask Questions Reusing the Cache

We first run `clean_up` to clear our cache (Good practice for CAGs).

Next, we convert our questions into tokens in `input_ids_q1` , then appended to the knowledge context stored in `ronan_cache` .

Finally, we call `generate` to produce the answer, decoding the final result with `tokenizer.decode` .

```
question1 = "Who is Ronan Takizawa?"
clean_up(ronan_cache, origin_len)
input_ids_q1 = tokenizer(question1 + "\n", return_tensors="pt").input_ids.to(dev
gen_ids_q1 = generate(model, input_ids_q1, ronan_cache)
answer1 = tokenizer.decode(gen_ids_q1[0], skip_special_tokens=True)
```

```
print("Q1:", question1)
print("A1:", answer1)
```

You should expect a response like this:

```
Q1: Who is Ronan Takizawa?
A1: Answer: Ronan Takizawa is an ambitious and accomplished
tech enthusiast. He has a diverse skill set in
software development, AI/ML...
```

Now we will save the cache to disk then reload it to prove that the cache persists for multiple sessions.

```
# Save the cache to disk
clean_up(ronan_cache, origin_len)
cache_dir = "cag_cache"
os.makedirs(cache_dir, exist_ok=True)

# Save the KV cache
torch.save(ronan_cache, os.path.join(cache_dir, "ronan_knowledge.cache"))

# Load cache to prove context is preserved for multiple sessions
loaded_cache = torch.load(os.path.join(cache_dir, "ronan_knowledge.cache"))

question3 = "What technologies has he worked with?"
input_ids_q3 = tokenizer(question3 + "\n", return_tensors="pt").input_ids.to(dev
gen_ids_q3 = generate(model, input_ids_q3, loaded_cache)
answer3 = tokenizer.decode(gen_ids_q3[0], skip_special_tokens=True)
```

You should get a response tailored to the context again.

## Conclusion

**Cache-augmented generation (CAG)** simplifies AI architectures by storing small knowledge bases directly within a model's context window, eliminating the need for retrieval loops in RAG and reducing latency. This approach enhances response speed and improves the responsiveness of an LLM with external knowledge. By leveraging CAG, developers can streamline their AI systems for faster and more efficient knowledge integration, particularly for tasks with stable, compact datasets.

Retrieval Augmented Gen | Retrieval Augmented | LLM

GoPenAI

## Published in GoPenAI

3.6K followers · Last published 1 day ago

Follow

Where the ChatGPT community comes together to share insights and stories.

## Written by Ronan Takizawa

378 followers · 21 following

Follow

Tech Content Creator (100k+ followers), 1x Acquired Founder,
https://www.linkedin.com/in/ronantakizawa/

## More from Ronan Takizawa and GoPenAI